Parallel Computing (BCS702)

1. **Write a OpenMP program to sort an array on n elements using both sequential and parallel mergesort (using Section). Record the difference in execution time.**

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void merge(int a[],int l,int m,int r){
    int i=l,j=m+1,k=0,t[r-l+1];
    while(i<=m&&j<=r)
        t[k++]=a[i]<=a[j]?a[i++]:a[j++];
    while(i<=m)
        t[k++]=a[i++];
    while(j<=r)
        t[k++]=a[j++];
    for(i=l,k=0;i<=r;i++,k++)
        a[i]=t[k];
}

void sort(int a[],int l,int r,int p){
    if(l<r){
        int m=(l+r)/2;
        if(p){
            #pragma omp parallel sections{
                #pragma omp section
                sort(a,l,m,1);
                #pragma omp section
                sort(a,m+1,r,1);
            }
        }
        else{
            sort(a,l,m,0);
            sort(a,m+1,r,0);
        }
        merge(a,l,m,r);
    }
}

int main(){
    int n=100000,a[n],b[n];
    double s,e;
    for(int i=0;i<n;i++)
        a[i]=b[i]=rand()%1000;

    s=omp_get_wtime();
    sort(a,0,n-1,0);
    e=omp_get_wtime();
    printf("Sequential Merge Sort Time: %f seconds\n",e-s);

    s=omp_get_wtime();
    sort(b,0,n-1,1);
    e=omp_get_wtime();
```

```
    printf("Parallel Merge Sort Time: %f seconds\n",e-s);
}
```

Create: gedit prg1.c

Compile: gcc -fopenmp prg1.c -o prg1

Run: export OMP_NUM_THREADS=4 && ./prg1

---

2. **Write an OpenMP program that divides the Iterations into chunks containing 2 iterations, respectively (OMP_SCHEDULE=static,2). Its input should be the number of iterations, and its output should be which iterations of a parallelized for loop are executed by which thread. For example, if there are two threads and four iterations, the output might be the following:**
    a) **Thread 0 : Iterations 0 – 1**
    b) **Thread 1 : Iterations 2 – 3**

```
#include <stdio.h>
#include <omp.h>
int main(){
    int n;
    printf("Enter number of iterations: ");
    scanf("%d",&n);
    int t[n];

    #pragma omp parallel for schedule(static,2)
    for(int i=0;i<n;i++)
        t[i]=omp_get_thread_num();

    for(int i=0;i<n;i+=2)
      printf("Thread %d : Iterations %d -- %d\n",t[i],i,i+1<n?i+1:i);
}
```

Create: gedit prg2.c

Compile: gcc -fopenmp prg2.c -o prg2

Run: export OMP_NUM_THREADS=4 && ./prg2

---

3. **Write a OpenMP program to calculate n Fibonacci numbers using tasks.**

```
#include <stdio.h>
#include <omp.h>
int fib(int n){
    if(n<2)
        return n;
    int x,y;
    #pragma omp task shared(x)
    x=fib(n-1);
    #pragma omp task shared(y)
    y=fib(n-2);
    #pragma omp taskwait
    return x+y;
}
```

```
int main(){
    int n;
    printf("Enter number of Fibonacci terms: ");
    scanf("%d",&n);
    printf("Fibonacci Series:\n");

    #pragma omp parallel
    #pragma omp single
    {
        for(int i=0;i<n;i++)
            printf("%d ",fib(i));
    }
    printf("\n");
}
```

Create: gedit prg3.c

Compile: gcc -fopenmp prg3.c -o prg3

Run: export OMP_NUM_THREADS=4 && ./prg3

---

4. **Write a OpenMP program to find the prime numbers from 1 to n employing parallel for directive. Record both serial and parallel execution times.**

```
#include <stdio.h>
#include <omp.h>

int prime(int n){
    if(n<2)
        return 0;
    for(int i=2;i*i<=n;i++)
        if(n%i==0)return 0;
    return 1;
}
int main(){
    int n;
    printf("Enter the value of n: ");
    scanf("%d",&n);

    int a[n+1],b[n+1];
    double s,e;

    printf("\nPrime numbers from 1 to %d:\n",n);
    for(int i=1;i<=n;i++)
        a[i]=prime(i);
    for(int i=1;i<=n;i++){
        if(a[i])
            printf("%d ",i);
    }
    printf("\n");
    s=omp_get_wtime();
    for(int i=1;i<=n;i++)
        b[i]=prime(i);
```

```
    e=omp_get_wtime();
    printf("Serial Time: %f seconds\n",e-s);

    s=omp_get_wtime();
    #pragma omp parallel for
    for(int i=1;i<=n;i++)
            b[i]=prime(i);
    e=omp_get_wtime();
    printf("Parallel Time: %f seconds\n",e-s);
}
```

Create: gedit prg4.c

Compile: gcc -fopenmp prg4.c -o prg4

Run: export OMP_NUM_THREADS=4 && ./prg4

---

5. Write a MPI Program to demonstration of MPI_Send and MPI_Recv.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc,char *argv[]){
    int rank,n=100;
    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);

    if(rank==0){
        printf("Process 0 sending number %d to process 1\n",n);
        MPI_Send(&n,1,MPI_INT,1,0,MPI_COMM_WORLD);
    }
    else if(rank==1){
        MPI_Recv(&n,1,MPI_INT,0,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
        printf("Process 1 received number %d from process 0\n",n);
    }
    MPI_Finalize();
}
```

Create: gedit prg5.c

Compile: mpicc prg5.c -o prg5 OR mpirun --oversubscribe -np 2 ./prg5

Run: mpirun -np 2 ./prg5

---

6. Write a MPI program to demonstration of deadlock using point to point communication and avoidance of deadlock by altering the call sequence.

```
#include <stdio.h>
#include <mpi.h>
#define MODE 2

int main(int argc,char *argv[]){
```

```
    int rank,n=123;
    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);

    if(MODE==1){
        printf("Process %d waiting to receive from Process %d...\n",rank,1-rank);
        MPI_Recv(&n,1,MPI_INT,1-rank,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
        MPI_Send(&n,1,MPI_INT,1-rank,0,MPI_COMM_WORLD);
    }
    else if(rank==0){
        MPI_Send(&n,1,MPI_INT,1,0,MPI_COMM_WORLD);
        MPI_Recv(&n,1,MPI_INT,1,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
        printf("Process 0 received back number: %d\n",n);
    }
    else{
        MPI_Recv(&n,1,MPI_INT,0,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
        MPI_Send(&n,1,MPI_INT,0,0,MPI_COMM_WORLD);
        printf("Process 1 received and sent back number: %d\n",n);
    }

    MPI_Finalize();
}
```

Create: gedit prg6.c

Compile: mpicc prg6.c -o prg6

Run: mpirun -np 2 ./prg6

---

7. Write a MPI Program to demonstration of Broadcast operation.

```
#include <stdio.h>
#include <mpi.h>

int main(int argc,char *argv[]){
    int rank,n;

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD,&rank);

    if(rank==0){
        n=50;
        printf("Process 0 broadcasting number 50 to all other processes.\n");
    }
    MPI_Bcast(&n,1,MPI_INT,0,MPI_COMM_WORLD);
    printf("Process %d received number %d\n",rank,n);
    MPI_Finalize();
}
```

Create: gedit prg7.c

Compile: mpicc prg7.c -o prg7

Run: mpirun -np 4 ./prg7

---

8. **Write a MPI Program demonstration of MPI_Scatter and MPI_Gather.**

```
#include <stdio.h>
#include <mpi.h>

int main(int argc,char *argv[]){
   int rank,size,x,a[100],g[100];

   MPI_Init(&argc,&argv);
   MPI_Comm_rank(MPI_COMM_WORLD,&rank);
   MPI_Comm_size(MPI_COMM_WORLD,&size);

   if(rank==0)
      for(int i=0;i<size;i++)a[i]=i*10;

   MPI_Scatter(a,1,MPI_INT,&x,1,MPI_INT,0,MPI_COMM_WORLD);
   x+=rank;
   MPI_Gather(&x,1,MPI_INT,g,1,MPI_INT,0,MPI_COMM_WORLD);

   if(rank==0){
      printf("Gathered data in root process:\n");
      for(int i=0;i<size;i++)
         printf("gathered_data[%d] = %d\n",i,g[i]);
   }
   MPI_Finalize();
}
```

Create: gedit prg8.c

Compile: mpicc prg8.c -o prg8

Run: mpirun -np 4 ./prg8

---

9. **Write a MPI Program to demonstration of MPI_Reduce and MPI_Allreduce (MPI_MAX, MPI_MIN, MPI_SUM, MPI_PROD).**

```
#include <stdio.h>
#include <mpi.h>

int main(int argc,char *argv[]){
   int rank,v,s,p,ma,mi,as,ap,am,ai;

   MPI_Init(&argc,&argv);
   MPI_Comm_rank(MPI_COMM_WORLD,&rank);
   v=rank+1;

   MPI_Reduce(&v,&s,1,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);
   MPI_Reduce(&v,&p,1,MPI_INT,MPI_PROD,0,MPI_COMM_WORLD);
   MPI_Reduce(&v,&ma,1,MPI_INT,MPI_MAX,0,MPI_COMM_WORLD);
   MPI_Reduce(&v,&mi,1,MPI_INT,MPI_MIN,0,MPI_COMM_WORLD);
```

```
   MPI_Allreduce(&v,&as,1,MPI_INT,MPI_SUM,MPI_COMM_WORLD);
   MPI_Allreduce(&v,&ap,1,MPI_INT,MPI_PROD,MPI_COMM_WORLD);
   MPI_Allreduce(&v,&am,1,MPI_INT,MPI_MAX,MPI_COMM_WORLD);
   MPI_Allreduce(&v,&ai,1,MPI_INT,MPI_MIN,MPI_COMM_WORLD);

   if(rank==0){
      printf("--- Results using MPI_Reduce (only root prints) ---\n");
      printf("Sum = %d\nProduct = %d\nMax = %d\nMin = %d\n",s,p,ma,mi);
   }

   printf("Process %d has value %d\n",rank,v);
   printf("Process %d sees (MPI_Allreduce): Sum=%d, Prod=%d, Max=%d, Min=%d\n",
         rank,as,ap,am,ai);

   MPI_Finalize();
}
```

Create: gedit prg9.c

Compile: mpicc prg9.c -o prg9

Run: mpirun -np 4 ./prg9